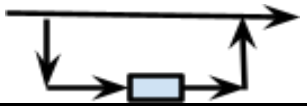
Práctica Nro. 2 Sintaxis

Objetivo: conocer cómo se define léxicamente un lenguaje de programación y cuáles son las herramientas necesarias para hacerlo.

Ejercicio 1: Complete el siguiente cuadro (p y p1 son producciones simbólicas):

Meta símbolos utilizados por		**Símbolo utilizado en Diagramas sintácticos**	**Significado**
BNF	**EBNF**		
palabra terminal	palabra terminal		Definición de un elemento terminal
No terminal	No terminal	rectángulo	Definición de un elemento no terminal
::=	::=	diagrama con rectángulos, óvalos y flechas	Metasímbolo
|	(|)	flecha que se divide en dos o más caminos	Selección de una alternativa
< p > < p1 >	{}		Repetición
	*		Repetición de 0 o más veces
	+		Repetición de 1 o más veces
	[]		Selección

Ejercicio 2: ¿Cuál es la importancia de la sintaxis para un lenguaje? ¿Cuáles son sus elementos?

La sintaxis permite escribir programas correctos y válidos sintácticamente. Establece reglas para combinar componentes básicas, y formar sentencias y programas. Los elementos que la componen son: alfabeto o conjunto de caracteres, identificadores, operadores, comentarios, palabras claves y reservadas.

Ejercicio 3: ¿Explique a qué se denomina regla lexicográfica y regla sintáctica?

- Regla lexicográfica: Define cómo se forman los símbolos básicos del lenguaje [tokens] (identificadores, números, palabras clave), es decir, qué secuencias de caracteres son válidas. Por ejemplo, un identificador debe empezar con letra y no con número. **"Palabras válidas"**.

- Regla sintáctica: Define cómo se combinan esos símbolos para formar estructuras válidas (expresiones, sentencias, programas), es decir, cómo se organizan los tokens. Ej: una asignación debe tener forma *variable = expresión;* **"Cómo armar oraciones con esas palabras"**.

Ejercicio 4: ¿En la definición de un lenguaje, a qué se llama palabra reservadas? ¿A qué son equivalentes en la definición de una gramática? De un ejemplo de palabra reservada en el lenguaje que más conoce. (Ada,C,Ruby,Python,..)

Las palabras reservadas son palabras que tienen un significado fijo dentro del lenguaje y no pueden usarse como identificadores (nombres de variables, métodos, etc.). Por ejemplo, en Java: if, else, while… etc.

Ejercicio 5: Dada la siguiente gramática escrita en BNF:

a- Identifique las componentes de la misma.

G= (N, T, S, P) → *Gramática (reglas finitas que definen un conjunto infinito de posibles sentencias válidas).*

N = {<numero_entero>, <digito>} → No terminales

T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9} → Terminales

S = <numero_entero> → Símbolo distinguido de la gramática que pertenece a N

P = {<numero_entero>::=<digito><numero_entero>|<numero_entero><digito>|<digito>

<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 } → Producciones

b- Indique porqué es ambigua y corríjala.

La gramática es ambigua porque una misma cadena puede tener más de un árbol de derivación. Por ejemplo, dado el número entero "12" (cadena) puedo derivarlo como *<digito><numero_entero> o <numero_entero><digito>, puesto que está utilizando recursión por derecha y por izquierda a la vez.*

```
<numero_entero>
     /      \
<digito>   <numero_entero>
   |            |
   1          <digito>
                |
                2
```

```
              <numero_entero>
                   /      \
<numero_entero>   <digito>
     |              |
<digito>            2
     |
     1
```

Si elimino una de las producciones recursivas se soluciona:

P = {<numero_entero>::=<digito><numero_entero>|<digito>

<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9}

Ejercicio 6: Defina en BNF (Gramática de contexto libre desarrollada por Backus-Naur) la gramática para la definición de una palabra cualquiera.

G = (N, T, S, P)

N = {<palabra>,<letra>}

T = {a,b...,z}

S = <palabra>

P = {<palabra> ::= <letra>|<letra><palabra>

<letra> ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | y | z}

Ejemplo para la cadena "pan"

<palabra> => <letra><palabra>

=> p <palabra>

=> p <letra><palabra>

=> pa <palabra>

=> pa <letra>

=> pan

Ejercicio 7: Defina en EBNF la gramática para la definición de números reales. Inténtelo desarrollar para BNF y explique las diferencias con la utilización de la gramática EBNF.

EBNF → *G = (N, T, S, P)*

N = {<real>,<digito>}

T = {1,2,3,4,5,6,7,8,9,+,-,.}

S = <real>

P = { <real> ::= [(+|-)]{<digito>}[. {<digito>}*]

<digito> ::= (1|2|3|4|5|6|7|8|9|0) }

BNF → *G = (N, T, S, P)*

N = {<real>,<digito>,<entero>}

T = {1,2,3,4,5,6,7,8,9,+,-,.}

S = <real>

P={<real>::= <signo><entero>|<signo><entero>.<entero>|<entero>|<entero>.<entero>

<entero> ::= <digito> | <digito><entero>

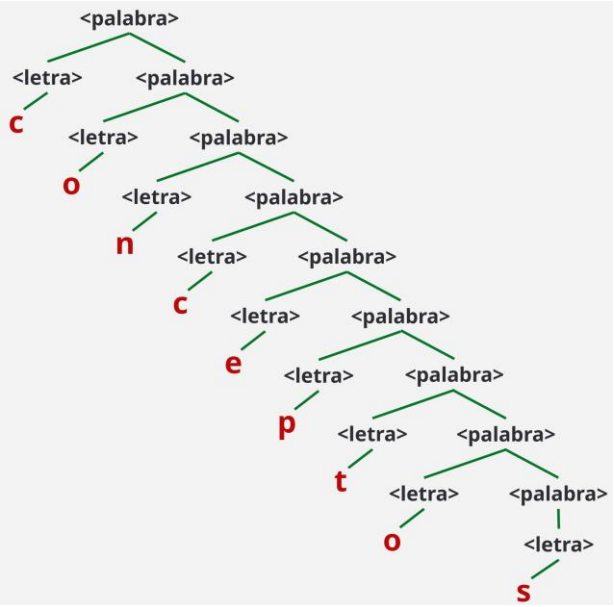
<signo> ::= +|-

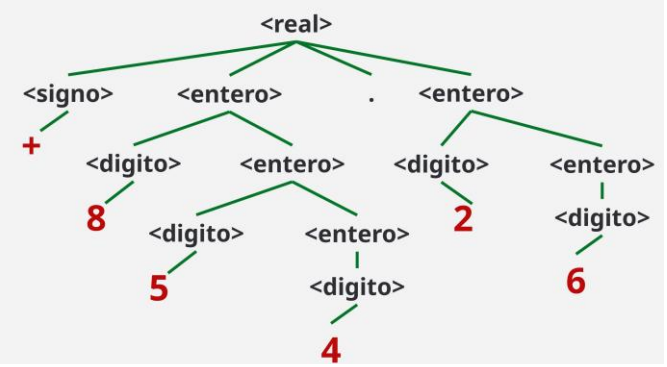
<digito> ::= (1|2|3|4|5|6|7|8|9|0) }

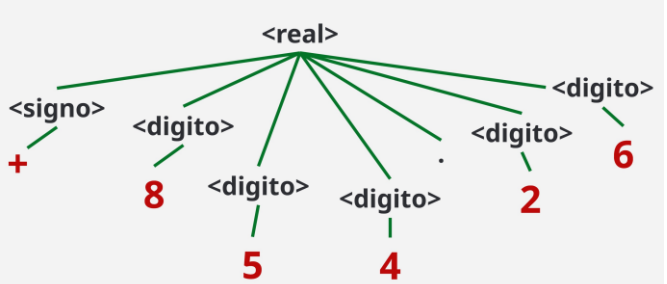
Ejercicio 8: Utilizando la gramática que desarrolló en los puntos 6 y 7, escriba el árbol sintáctico de:

a. Conceptos
b. Programación
c. 1255869
d. 854,26
e. Conceptos de lenguajes

Voy a hacer sólo a. y d. (son todos iguales). El punto e. no lo puedo hacer porque no definí el espacio, además todas las palabras las puedo desarrollar en minúscula, puesto que tampoco contemplo las mayúsculas en la gramática.

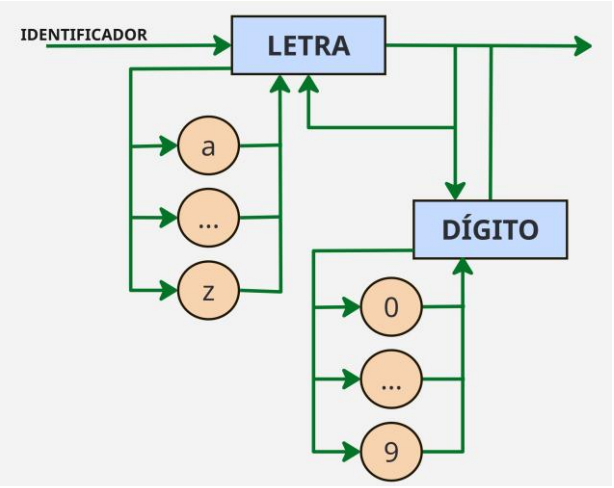






Ejercicio 9: Defina utilizando diagramas sintácticos la gramática para la definición de un identificador de un lenguaje de programación. Tenga presente como regla que un identificador no puede comenzar con números.

Suponiendo que el identificador sólo puede tener letras y números, y no comenzar con un número (no tengo en cuenta los caracteres especiales, y tampoco las mayúsculas para simplificar) nos quedaría:



Ejercicio 10:

a) Defina con EBNF la gramática para una expresión numérica, dónde intervienen variables y números. Considerar los operadores +, -, * y / sin

orden de prioridad. No considerar el uso de paréntesis.

G = (N,T,S,P)

N = {<expresión>,<variable>,<numero>,<operador>,<digito>,<letra>}

T = {0, 1, 2, …, 9, a, b, …, z, "+", "-", "*", "/"}

S = <expresión>

P = {<expresion> ::= (<variable>|<numero>){<operador>(<variable>|<numero>)}*

<variable> ::= {letra}*

<numero> ::= [(+|-)]{<digito>}[. {<digito>}*]

<operador> ::= (+ | - | * | /)

<digito> ::= (0 | 1 | … | 9)

<letra> ::= (a | b | … | z) }

b) A la gramática definida en el ejercicio anterior agregarle prioridad de operadores.

c) Describa con sus palabras los pasos y decisiones que tomó para agregarle prioridad de operadores al ejercicio anterior.

Ejercicio 11: La siguiente gramática intenta describir sintácticamente la sentencia for de ADA, indique cuál/cuáles son los errores justificando la respuesta.

Falta declarar la gramática G = (N,T,S,P)

N= {<sentencia_for>, <bloque>, <variable>, <letra>, <cadena>, <digito>, <otro>, <operacion>, <llamada_a_funcion>, <numero>, <sentencia> }

Faltan las terminales T = {a,…,z,A,…,Z,0,1,…,9

Falta S = <sentencia_for>

P= { <sentencia_for>::= for (i= IN 1..10) loop <bloque> end loop;

<variable>::= <letra> | <cadena>

<cadena>::= { (<letra> | <digito> | <otro> **sin definir**) }+

<letra>::=(a | .. | z | A | .. | Z)

<digito>::= (1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0)

<bloque>::= <sentencia> | <sentencia> <bloque> | <bloque> <sentencia> **hay ambiguedad**;

<sentencia>::= <sentencia_asignacion> | <llamada_a_funcion> | <sentencia_if> | <sentencia_for> | <sentencia_while> | <sentencia_switch> **sin definir** }

Ejercicio 12: Realice en EBNF la gramática para la definición un tag div en html 5. (Puede ayudarse con el siguiente enlace (https://developer.mozilla.org/es/docs/Web/HTML/Elemento/div)

G = (N,T,S,P)

N = { ,<letra>,<contenido><digito><palabra>}

T = {a,…z,A,…,Z,0,…,9,<,>,/}

S =

P = { ::= “ ” [<contenido>] “ ”

<contenido> = { <palabra> | }

<palabra> ::= <letra> { (<letra> | <digito>) }*

<letra> ::= (a | … | z | A | … | Z)

<digito> ::= (0 | 1 | … | 9) }

Ejercicio 13: Defina en EBNF una gramática para la construcción de números primos. ¿Qué debería agregar a la gramática para completar el ejercicio?

Ser primo es una propiedad aritmética, no sintáctica. Para que la gramática genere solo números primos, es necesario agregar restricciones semánticas o un mecanismo de validación externo, ya que el hecho de que sea primo no puede expresarse únicamente con EBNF. La gramática que se usa sería básicamente la misma que la de los números enteros, después se le agrega la restricción semántica.

Ejercicio 14: Sobre un lenguaje de su preferencia escriba en EBNF la gramática para la definición de funciones o métodos o procedimientos (considere los parámetros en caso de ser necesario)

G = (N,T,S,P)

N = {<funcion>,<lista_param>,<parametros>,<palabra>,<letra>,<digito>}

T = {a,…z,A,…,Z,0,…,9, “(“, “)”, “{“, “}”, “,” }

S = <funcion>

<funcion> = "function" <palabra> "(" <parametros> ")" "{ }"

<parametros> = [<lista_param>]

<lista_param> = <palabra> { "," <palabra> }

<palabra> ::= <letra> { (<letra> | <digito>) }*

<letra> ::= (a | … | z | A | … | Z)

<digito> ::= (0 | 1 | … | 9) }